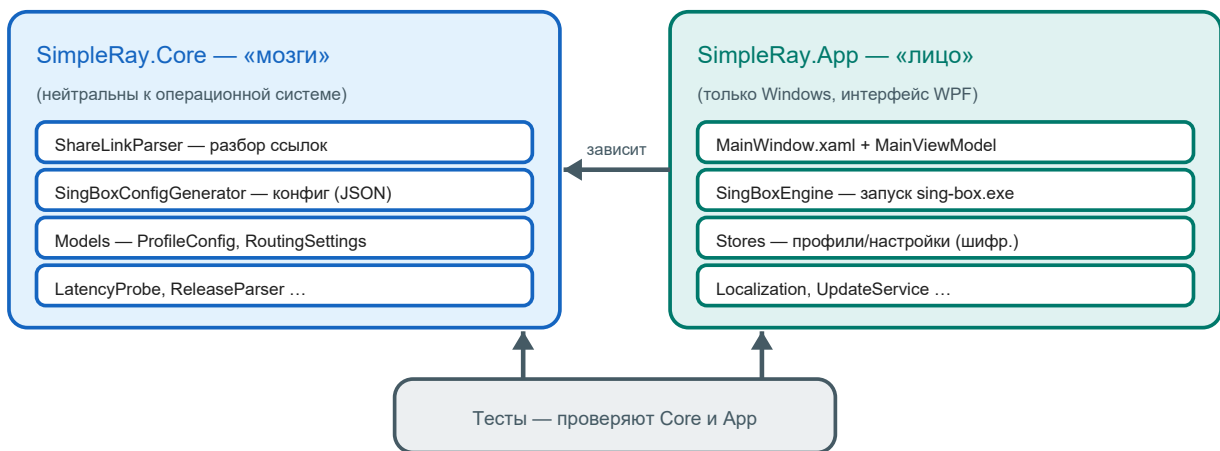


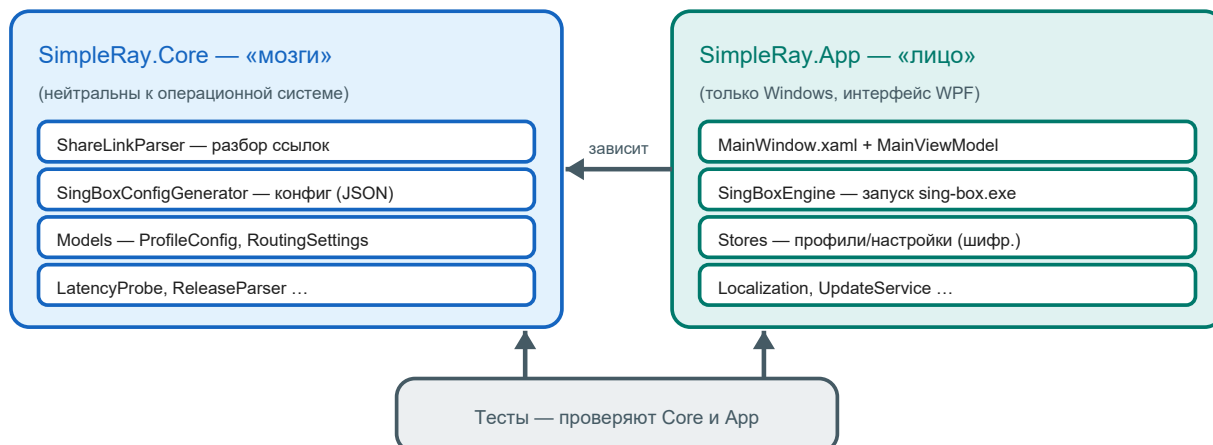
SimpleRay изнутри

Как устроена программа — иллюстрированный разбор для новичка



VPN-клиент на C# / .NET 8 (WPF) поверх движка sing-box

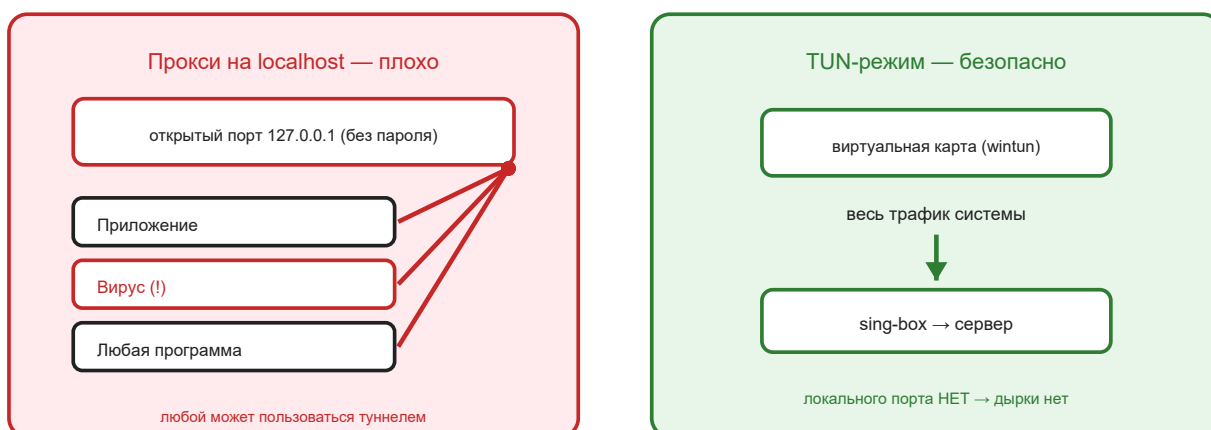
1. Из чего собрана программа



Проект делится на два подпроекта. Core — чистая логика (разбор ссылок, генерация конфигурации, модели данных), которая ничего не знает про Windows и окна. App — интерфейс на WPF, запуск процессов, работа с файлами.

Такое разделение (separation of concerns — «разделение ответственности») позволяет проверять «мозги» тестами без окна и переиспользовать их — например, завтра под Android. App зависит от Core, но не наоборот.

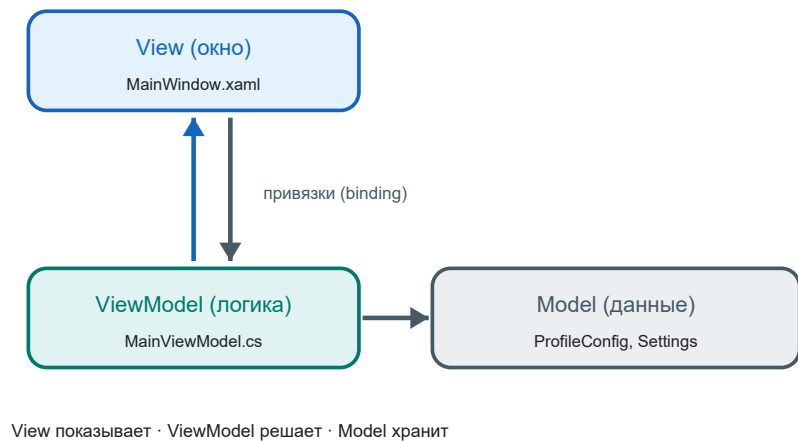
2. Главная идея безопасности: TUN, а не прокси



Многие клиенты открывают локальный прокси-порт на 127.0.0.1 — часто без пароля, и любая программа (даже вредоносная) может им пользоваться как бесплатным туннелем.

SimpleRay вместо этого поднимает виртуальную сетевую карту (TUN) через wintun: весь трафик системы идёт через неё, а локального порта нет вовсе. Уязвимость закрыта не заплаткой, а конструкцией. Тест AntiLeakTests следит, чтобы входящий прокси никогда не появился в конфиге.

3. Интерфейс: WPF и паттерн MVVM

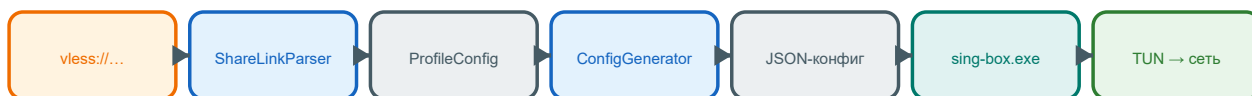


Окно разложено на три слоя. View (MainWindow.xaml) только показывает. ViewModel (MainViewModel.cs) содержит логику. Model — данные из Core.

Их связывают привязки (binding): надпись на кнопке берётся из свойства ViewModel и обновляется сама, когда свойство меняется. Кнопки — это команды (RelayCommand) с методами «что делать» и «можно ли нажимать».

4. Путь профиля: от ссылки до туннеля

Путь профиля: строка → объект → JSON → движок → туннель

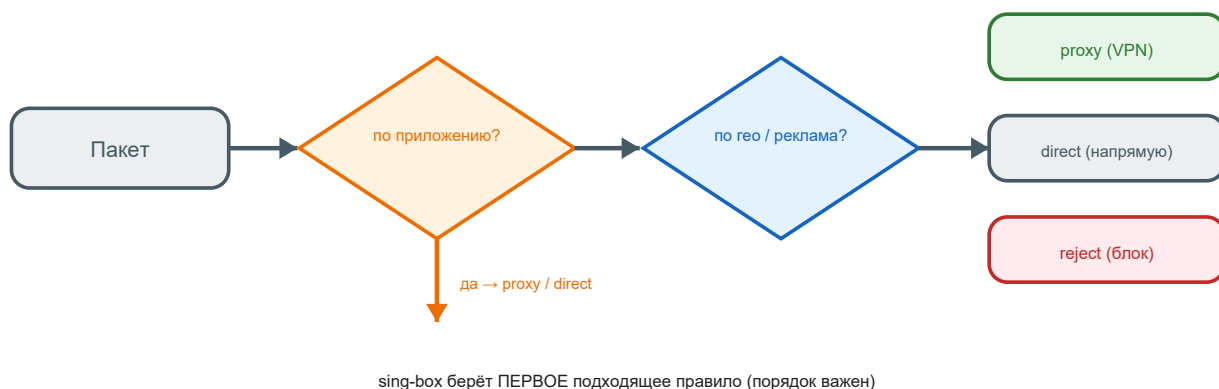


Core только ОПИСЫВАЕТ намерение (JSON). Сетевую работу делает sing-box.

Главный сценарий-«конвейер». Ссылка вида `vless://...` разбирается в объект `ProfileConfig`. При нажатии «Подключить» генератор превращает его (плюс настройки маршрутизации) в большой JSON для `sing-box`, и движок запускает `sing-box.exe` с этим конфигом.

Ключевая мысль: Core только описывает намерение в JSON; реальную сетевую работу делает `sing-box`. Правильность JSON проверяется командой `sing-box check` ещё до запуска.

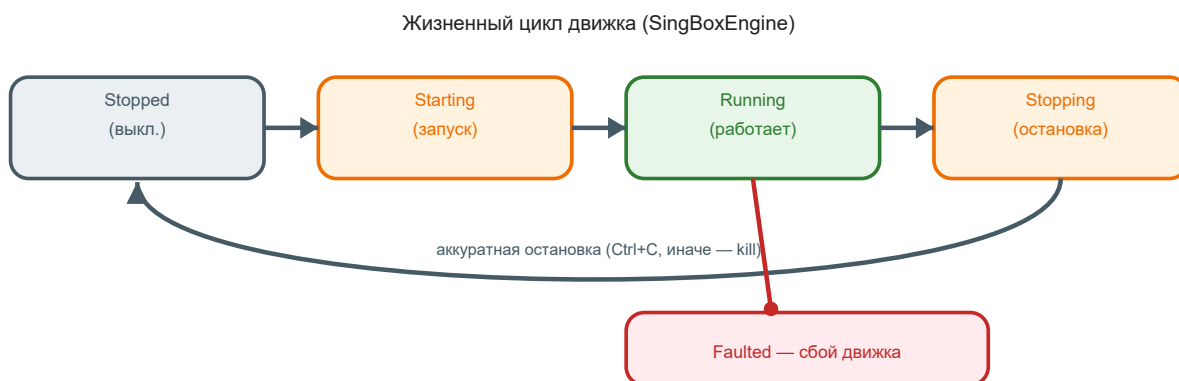
5. Маршрутизация: куда пойдёт пакет



VPN не обязан гнать через сервер весь трафик. Правила решают судьбу каждого пакета: сначала правила по приложению (по имени процесса), затем по гео (geoip/geosite — какой адрес к какой стране относится) и блокировка рекламы, иначе — итоговое действие режима (proxy / direct).

sing-box берёт первое подходящее правило, поэтому порядок важен — правила по приложениям стоят раньше гео-правил (это проверяет тест AntiLeakTests). Ещё есть группа каналов: при сбое sing-box сам переключается на живой сервер (failover — «отказоустойчивое переключение»).

6. Движок: управление процессом sing-box



SimpleRay не делает сеть сам — он дирижирует дочерним процессом sing-box. За движком стоит контракт IVpnEngine (Start/Stop, состояние, лог); под Windows его реализует SingBoxEngine, а завтра под Android будет другая реализация того же контракта.

Состояния сменяются: выключен → запуск → работает → остановка. Отдельно — сбой (Faulted). Аккуратная остановка важна: просто «убить» процесс нельзя — sing-box не успеет убрать сетевые маршруты, и можно остаться без интернета. Поэтому сначала посылается вежливый сигнал (Ctrl+C), и только при неудаче — принудительное завершение.

7. Обновления с откатом

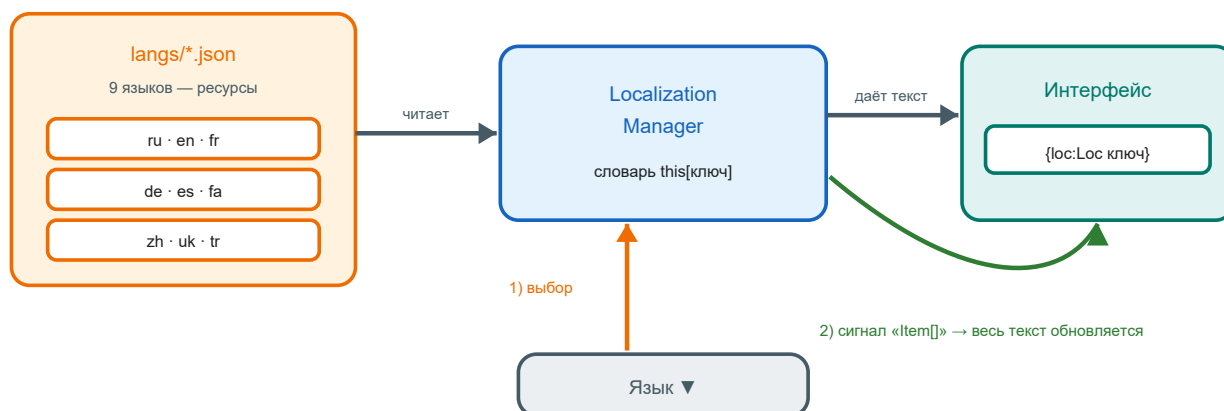


Программа обновляет себя безопасно: спрашивает у GitHub последнюю версию, качает архив только с согласия пользователя и сверяет его контрольную сумму SHA256 (защита от порчи при передаче).

Переписать себя «на ходу» нельзя, поэтому копия-обновлятор ждёт закрытия программы, делает резервную копию, подменяет файлы и запускает новую версию. Если та сразу падает — обновлятор восстанавливает старую. «Не завёлся мотор — вернули прежний».

8. Языки интерфейса: перевод «на лету»

Смена языка «на лету» — без перезапуска окна



Программа говорит на девяти языках, и переключение происходит без перезапуска окна. Все надписи лежат не в коде, а в отдельных файлах-словарях `langs/*.json` (по одному на язык), встроенных в программу как ресурсы.

В окне вместо готового текста стоит ссылка на ключ — запись `{loc:Лос ключ}`. Её обслуживает `LocalizationManager`: он работает как словарь «ключ → перевод». Когда вы выбираете язык в списке, менеджер подаёт сигнал `«Item[]»`, и все привязки перечитывают текст сами — интерфейс мгновенно меняет язык.

Тест `LocalizationTests` следит, чтобы во всех девяти файлах был одинаковый набор ключей, не было пустых строк и совпадали подстановки вида `{0}`. Поэтому любую новую надпись приходится добавлять сразу во все языки — забыть перевод не получится.

9. Хранение секретов: DPAPI и атомарная запись

Секреты на диске: шифрование DPAPI и атомарная запись

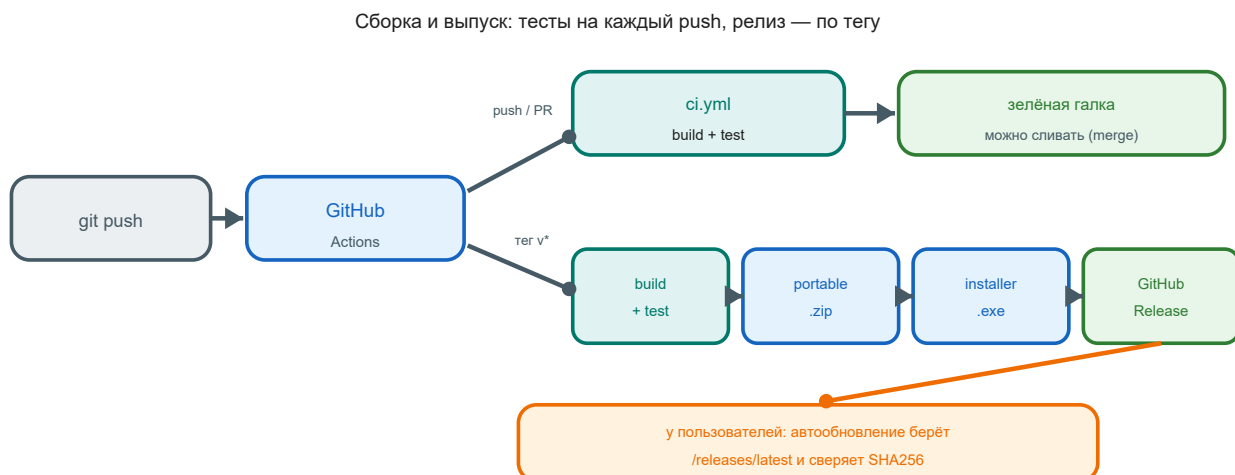


Профили и настройки хранятся в вашей папке данных. Пароли и адреса серверов — это секреты, поэтому файл шифруется средствами Windows — DPAPI (ProtectedData). Ключ шифрования привязан к вашей учётной записи, так что другой пользователь или другой компьютер файл не прочитают.

Записывается файл атомарно: сначала данные пишутся во временный файл, затем одной операцией File.Replace он заменяет старый. Если питание пропадёт посреди записи — останется целый прежний файл, а не «половинка».

Формат помечен полем `schemaVersion` — это позволяет менять структуру в будущем и по-прежнему открывать старые файлы (обратная совместимость). Старый незашифрованный формат тоже ещё читается и молча переводится на новый.

10. Как собирается и выпускается программа



Каждый раз, когда код отправляется на GitHub (git push или Pull Request), сервер сам запускает рабочий процесс ci.yml: собирает проект и прогоняет все тесты. Красная галочка вместо зелёной сразу показывает, что что-то сломалось, — ошибку видно ещё до пользователя.

Выпуск версии запускается по тегу вида v0.2.0: процесс release.yml собирает и тестирует, делает portable-архив и установщик, а затем создаёт релиз на GitHub с этими файлами. Встроенная автообновлялка потом берёт последний релиз (/releases/latest) и сверяет контрольную сумму SHA256.

Так «сборка вручную» превращается в конвейер (CI/CD): человек только ставит тег, а всё остальное — сборку, тесты и публикацию — сервер делает одинаково и без ошибок.

Расшифровка сокращений

Технические сокращения, встречающиеся в проекте и в этом разборе.

Сокр.	Расшифровка	Что значит простыми словами
VPN	Virtual Private Network	виртуальная частная сеть — защищённый канал до сервера
TUN	network TUNnel (адаптер)	виртуальная сетевая карта; через неё идёт весь трафик
WPF	Windows Presentation Foundation	технология окон для Windows
XAML	eXtensible App Markup Language	язык разметки окон (похож на HTML)
MVVM	Model – View – ViewModel	раскладка интерфейса на три слоя
UI	User Interface	пользовательский интерфейс
JSON	JavaScript Object Notation	текстовый формат данных (конфиг sing-box)
DPAPI	Data Protection API	встроенное шифрование Windows под учётную запись
TLS	Transport Layer Security	шифрование соединения («замок» в браузере)
DNS	Domain Name System	«телефонная книга» интернета: имя → адрес
SHA256	Secure Hash Algorithm 256	«отпечаток» файла для проверки целостности
CI/CD	Continuous Integration / Delivery	авто-сборка, тесты и выпуск на сервере
i18n / L10n	internationalization / localization	перевод интерфейса на разные языки
PR	Pull Request	запрос на слияние изменений в общий код
SDK	Software Development Kit	набор инструментов разработчика
API	Application Programming Interface	как одна программа обращается к другой
DLL	Dynamic-Link Library	библиотека кода, подключаемая на лету (wintun.dll)
geoip / geosite	geo-IP / geo-site	базы соответствия «адрес/сайт → страна»
QR	Quick Response code	квадратный штрих-код
RTL	Right-To-Left	письмо справа налево (арабский, фарси)

Что почитать в коде дальше

- ProfileConfig.cs — простой объект-«коробка» с полями профиля.
- ShareLinkParser.cs — как строка превращается в объект.
- SingBoxConfigGenerator.cs — как объект превращается в JSON.
- MainViewModel.cs — «клей» логики окна.
- LocalizationManager.cs — как одна смена языка обновляет весь текст сразу.
- tests/ — каждый тест это пример «дано → ожидаем»; лучший способ понять код.

Полный текстовый разбор — в файле docs/GUIDE.ru.md. Сборка: dotnet build, тесты: dotnet test, запуск окна: dotnet run --project src/SimpleRay.App. Для настоящего подключения нужны права администратора (TUN) и рабочий сервер.